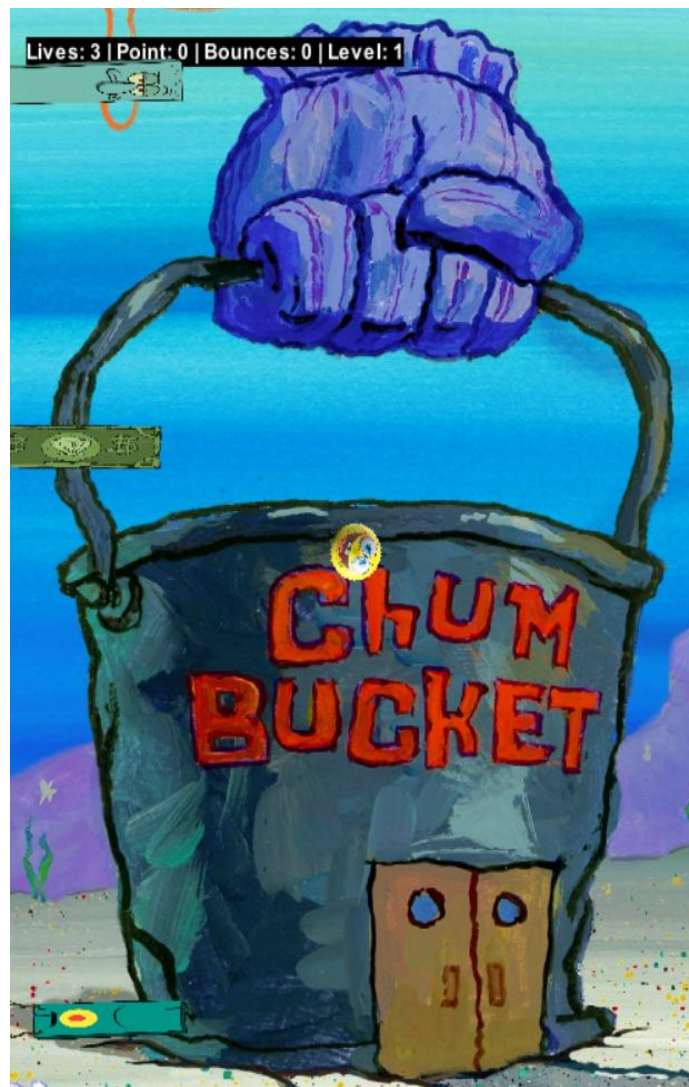




Ping - Compulsory Assignment #1

Handed-in by

1. Mohammad-Brousk Othman Ali
2. Steffan Palompon Lindhard Nissen
3. Simon Løager Kiilsgaard Henrichsen

Date: 18/09-2026

Table of Contents

1. Introduction.....	1
1.1. Background.....	1
1.3. Product vision	1
2. Pre-Game/Background.....	2
2.1. Project organization.....	2
2.2. Overall Project schedule	2
2.3. Initial product backlog.....	3
2.4. Architecture	4
3. Sprint 1	5
3.3. GUI.....	7
3.5. Implementation.....	7
3.5.1. Code examples.....	8
3.7. Sprint Review	10
3.8. Sprint Retrospective	11
7. Conclusion.....	11
References	12

1. Introduction

The project is a functional version of pong that works through the Greenfoot application. Pong is a classic game that resembles tennis, in that there are two players who control a paddle and where the goal is to make the opponent unable to return the ball, resulting in a point for the person who most recently sent the ball towards the opponents part of the “court”. In most pong games, the players are located on the left and right part of the screen, but in this project, the player-controlled paddle is at the bottom of the screen and as part of the mandatory parts of the assignment, there is a paddle that spawns between the top of the screen and the middle of the screen.

1.1. Background

We received a Greenfoot program, where you first encounter a welcome screen with instructions regarding how to get to the game state. When you press enter, a paddle is present at the bottom of the screen, a ball appears and then moves towards the bottom of the screen and that is about it. The code contains a bunch of empty methods that the group is then meant to implement and get to a functional state, so the game runs according to the parts of the assignment and with the (encouraged) possibility of a personal touch, to make it more fun. We all live quite far apart, but agreed that we would be doing most of the work at SEA, since the amenities there are beneficial for us (such as the screens where we could utilize Airtame) and the possibilities of being able to find help quickly from either lecturers or fellow students.

Our group consists of three members, who have various degrees of experience when it comes to working with code and programming. Because of this, we agreed that it would be beneficial for all of us to work on every part together, as the members with least experience would then be able to question how and why certain parts worked or were needed, and the more experienced members would also be challenging themselves and their understanding, when they had to elaborate on how parts worked.

For the report, we decided to all work on different parts and just consult each other, so we all knew what was in the report, and we would all be active participants.

1.3. Product vision

Our plan was to make a fast and fun arcade games based on the classic ping/pong game. We wanted to change the boring standard blocks and use a fun SpongeBob SquarePants theme instead.

We wanted a game where the player controls Plankton at the bottom of the screen to hit a flying SpongeBob. The game should get harder and faster as you play. We also wanted a

complete game experience, this means having a nice start screen with instructions, funny sound effects, and a clear game over screen when you lose.

2. Pre-Game/Background

2.1. Project organization

To work well together our group made a working agreement before we started coding. We organized our work by dividing the project's tasks into smaller pieces.

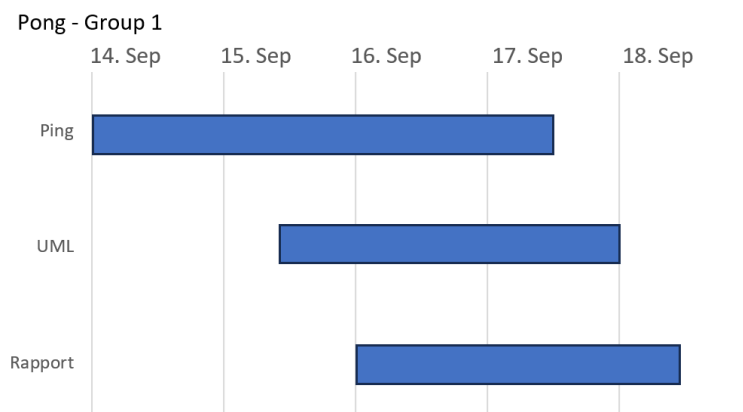
- coding and logic: Steffan, Simon and Mohammed were responsible for writing the main game logic. This included making the paddle move, fixing the bounce physics for the ball, and creating the speed up. Rather than assigning each feature exclusively to one person, we worked together on the programming. This allowed us to discuss problems and understand each other's solutions.
- graphics, audio: we all came with ideas, and were responsible for finding and scaling the SpongeBob themed images. This included the plankton, paddle, the money bill, and the background.

We made a simple timeline to make sure everything was done on time. The main coding had to be done first, so the design team could add the images later. Finally, we tested the game together to fix any errors before writing this report.

We agreed that all code must be compiled without errors before saving. We also wanted the graphics to look clean and transparent, with no weird trails on the screen, so the game looks professional and feels good to play.

2.2. Overall Project schedule

On the right is the Gantt diagram of our planned schedule for the project of the week. To manage time and ensure all deadlines were met the project was split into three main focus points from September 14 to September 18, which are: the Ping game, UML, and final project report writing.



2.3. Initial product backlog

The product backlog is a complete list of all the features and requirements we need to build for our game. This list is divided into mandatory tasks required to pass and optional tasks. Construct a single-player game, with the paddle at the floor and the ball will bounce off the ceiling and walls. The “left” and “right” keys should operate the paddle.

1. Add a self-moving paddle, which enters the world at a random position. The self-moving paddle moves horizontally. When the paddle moves out of a wall, a new paddle should show up at a random height on the other wall. Make sure the self-moving paddle is not created too near the gamers paddle.
2. When the ball hits a self-moving paddle from below it will bounce off the paddle. If the ball hits the self-moving paddle from above the ball will go right through the paddle.
3. Change the images of the game so it looks more nicely such as new images for the ball and the paddles.
4. Put sounds to the game for the ball hitting the paddle, ceiling, walls, and the floor (game over).
5. Each time the ball has been hit by the paddle 10 times, the speed of the ball is increased slightly.
6. Include a text field with the text “Game Level: 1” in the upper right corner of the game. Each time the speed increases, the game level increases too. Make sure the ball can pass in front of the text.

Optional Tasks (not needed to pass)

- a) Use an opening screen with an image and maybe some instructions. Get the screen to be shown a certain amount of time, or until the player hits the “enter”-key.
- b) Display a game-over image and play a sound when the game is over. Get the screen to be shown a certain amount of time, or until the player hits the “enter”-key and then restart the game.
- c) Add different sizes of self-moving paddles.
- d) Include the second computer-controlled paddle at the ceiling. Each time the player-controlled paddle is returning the ball, the computer-controlled paddle should start moving towards the point of contact. When the computer-controlled paddle hits the ball, the paddle stays in position awaiting the returning ball from the player-controlled paddle.
- e) Include two score fields instead counting the number of games won by the player and the computer (see the image of the original game).
- f) Make the computer-controlled paddle better working. Let it seek back towards the middle position after it has returned a ball. If the player returns the ball before the middle point has been reached, the paddle is off course moving towards the point of ball contact.
- g) And whatever crosses your mind to make the game more fun...

3. Sprint 1

We decided to keep a logbook of the days, so it could help us in future projects, as we expect to run into similar issues in the future. We anticipate that it will be easier to deal with problems once we have a detailed log of problems that occurred and how we handled them.

Monday 14/09-2026

Having started the first part of the assignment on Friday, today we set out to work our way through the backlog of mandatory elements, so we would be able to focus on the quality-of-life parts later, once the foundation was solid and set.

We began by studying the code together, so we would be on the same page and have a common understanding of the code, and be able to give input or question any parts if needed. One of our first decisions was to think of a theme for the project. We agreed on a theme of SpongeBob which would be amusing and have options for a game that is meant to be fun. We ran into an issue, where the size of the paddles did not reflect the hitboxes, but since collision was not otherwise broken at the time, we decided that the flow of our work should take precedence. This decision proved to be a frugal one for this group, as we made our way through the first 5 mandatory parts of the assignment, and got started on the basic parts of the last 2.

After a short break, we returned to talk about what we had accomplished thus far and what we wanted to do with the rest of the day, as well as plan our goals for tomorrow.

Tuesday 15/09-2026

Today was marked by a lot of challenges in various sizes.

We encountered a problem, where the code wouldn't compile after a basic if/else statement. We tried experimenting with various solutions, we deleted the additions and the program suddenly compiled with no new changes or additions.

We wanted to make the game more fun to look at when you lose the game, so we tried adding a game over screen, in the same way we had added other background pictures before. We ran into a FileNotFoundException, where our game would crash and the error message told us that the file could not be located. As we tried troubleshooting, we unfortunately downloaded another .webp picture, which (in hindsight obviously) gave us the same error. We spent roughly an hour trying to get it to work, due to us not understanding the problem and reading the error message as if the file itself could not be located. Our solution was to open the picture in a different program, take a picture of it and then save it as a .png file, which we knew was accepted as a file type in Greenfoot.

A member of a different group made us aware of the problem with our hitbox being since our bottom paddle was actually larger than it seemed, so the y-axis bounce was actually as it should be - on a technical level at least. We added another stipulation to the bounce check,

where it would only accept collision when the ball was beneath a certain point on the y-axis as well.

Later in the day, we put in some more work on the UML diagram.

When the day was nearing the end, we agreed to work from home tomorrow, but stayed on a Discord call from 9-13 again.

Wednesday 16/09-2026

We started the day by looking at minor issues we had noticed and/or were able to provoke, such as getting the ball to get stuck in the top of the screen if it happened to clip through the ceiling and worked on a fix for that. We also spent some time discussing if we all understood it the same way, for example part a:

“Use an opening screen with an image and maybe some instructions. Get the screen to be shown a certain amount of time, or until the player hits the “enter”-key.” We wanted to make sure we all agreed that the “, or until the...” part meant that our current solution was acceptable both to us, but also for the actual meaning and spirit of the assignment.

Once we had worked on some minor fixes for the code, we implemented the random size of the middle paddle. We agreed to let the height be static, but the width is affected by a random number, as it looked better.

While working on the report, we discussed the possibility of making the top paddle (part d of the optional tasks) player controllable, which would make the game more fun and be an additional solution to optional task g: “And whatever crosses your mind to make the game more fun...”

Thursday 17/09-2026

We started the day by discussing ideas and thoughts we had since yesterday regarding the report and the program itself. We talked about an imported Greenfoot class (the Gif image class) and whether we should keep it or revert back to solely using images. We decided that since the Gif class itself is massive and we would need to spend a lot of time reading in order to actually understand it, our time would be better spent on parts we had created and could improve on instead, so we removed the Gif class. We discussed how we felt the project had evolved and how we each had experienced the group and the current result of our work. We worked on improvements and talked about which parts of the code we would like to include in the report as code examples.

Friday 18/09-2026

On the final day of the project, we spent most of our time trying to make quality of life improvements to the game and make the report legible and understandable to people who haven't been part of the group.

Most of the time was spent finishing parts of the report that we either weren't satisfied with or where the wording no longer matched the actual program or our intentions. We spent some

time including a couple of snippets of code, so we could explain some of the parts that were included to show some of our ideas for solutions or fixes for problems, like the collision issue.

We debriefed and talked about what we had learned from the assignment and from working together. We discussed what we want to take with us to the next project and if there are any changes that would be beneficial.

Once we had talked about what we could do individually as well, we thanked each other for the teamwork and for keeping both good spirits when some of the tasks had been irritating and to make sure everyone felt welcome in the group and as active participants.

3.3. GUI

The user interface was designed around a simple arcade game layout. The player-controlled paddle is positioned at the bottom of the screen. The player uses the A/left and D/right arrow keys to control the paddle. The ball is placed in the playing area and moves continuously. The computer-controlled paddle is positioned in the upper part of the game. The game level is displayed in the upper left corner. This allows the player to see how far they have progressed and provides an indication that the game is becoming more difficult. We chose the SpongeBob theme because we wanted to move away from the basic appearance of the original Pong game. The graphics were therefore changed to fit the theme, including the player paddle, computer paddle, ball, and background. An opening screen was also used to provide the player with instructions before the game starts.



Figure 4 Main menu screen

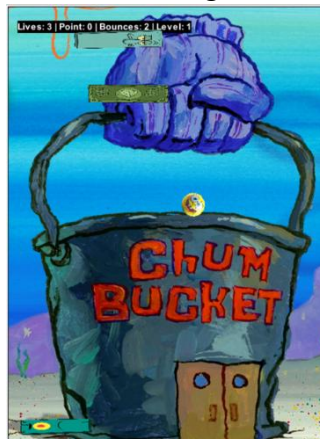


Figure 2 ingame screenshot



Figure 3 losing the game screenshot

3.5. Implementation

We carried out the implementation step by step. We first focused on the basic gameplay and then added additional functionality.

The player paddle was implemented so that it responds to the A/left and D/right arrow keys. The ball was made with continuous movement and simple collision detection.

The collision system was one of the parts that required the most adjustment. In particular, we had to look at the difference between the visible image of a paddle and its real collision area.

The middlePaddle was made with horizontal movement and randomized positioning. The game progression was made by counting successful hits from the player's or the computer-controlled paddle. Once the required number of hits was reached, the ball speed increased and the game level was updated and got harder. Graphics and sound effects were then added to improve the overall presentation.

3.5.1. Code examples

While testing the ball's bouncing mechanics, we ran into some issues. Because our paddle image 'PLANKton.png' included long antlers extending from the side, Greenfoot created a box that covered the image dimensions, acting as the initial hitbox, making it much wider than the actual paddle. Visually, the ball was bouncing off the paddles way too early before actually making contact. It looked unnatural since the 'isTouching()' was triggering while there was a noticeable empty space between the ball and paddle. To fix the oversized hitbox issue, we used Greenfoot's inspect function during play to see the exact coordinates. We paused the game right when 'isTouching()' returned true and checked the Y coordinates of both the ball and the paddle. By measuring the distance between the center of the paddle and the actual visual surface of the paddle, we added specific coordinate checks like (getY() > 617) for the bottom paddle and (getY() < 83) for the top paddle so the collision only triggered when the ball had reached the actual paddle, rather than hitting the invisible hitbox as seen in the picture.

```
private void checkBounceOffPaddle()
{
    if (isTouching(MiddlePaddle.class)) //checking MiddlePaddle
    {
        if (getRotation() > 180 && !hasBouncedVertically)
        {
            revertVertically(false);
        }
    }
    /**
     * Checks the players paddle and that the ball always bounces
     * And adjusted the hitbox for the player paddle if the ball is touching
     */
    else if (isTouching(Paddle.class) && (getY() > 617))
    {
        if (!hasBouncedVertically) //bounce off player paddle
        {
            revertVertically(true);
        }
    }
    else if (isTouching(TopPaddle.class) && (getY() < 83))
    {
        if (!hasBouncedVertically)
        {
            revertVertically(true);
        }
    }
    else
    {
        hasBouncedVertically = false; //reset bounce lock
    }
}
```



Figure 5 PLANKton.png

```
private void checkPaddleTouchWall() //once reached the right edge of the map, remove this one and spawn a new one
{
    if (getX() + width / 2 >= getWorld().getWidth())
    {
        PingWorld world = (PingWorld) getWorld();
        world.removeObject(this); //removing the instance from the world

        //randomizer for width of the self moving paddle, which gives the game different sizes of moving paddles
        int randomWidth = 100 + Greenfoot.getRandomNumber(70);
        world.addObject(new MiddlePaddle(randomWidth, 70), 0, Greenfoot.getRandomNumber(200) + 100);
    }
}
```

The method above from the MiddlePaddle class is how we handle the second mandatory task, where a paddle needs to spawn at a random location, but not too close to the bottom paddle. We created it with an if statement, where the first check is to see if the right side of the paddle is touching the right wall and if so, the object is called from the PingWorld and is then removed. After this has happened, a randomWidth variable is called, where - as the name implies - only the width is assigned a random value between 100 and 169 pixels as the random number is set to 70 and can include 0, but the +100 means it will always start from 100, so the middle paddle has a good chance to be different from the last one. After this, the object is summoned into the world at a somewhat random location on the left wall between y= 99 and y= 299, meaning it won't be at the very top of the screen and it won't spawn below the middle of the Pingworld, which would be y=350.

The purpose of 'trackBall()' was to simulate a computer-controlled paddle, to give the player an opponent. First, the code checks 'ball.getRotation() > 180' to determine if the ball is moving upward toward the 'TopPaddle' (computer-controlled paddle). This ensures the opponent only reacts when it is necessary instead of constantly chasing the ball while it travels down to the player's paddle. So, when approaching, the paddle gets the ball's horizontal position (targetX). We use 'paddleSpeed' as a threshold to prevent pixel jitters during the paddle's alignment. The paddle only moves left or right by using the 'setLocation()', keeping its vertical position fixed.

```
private void trackBall()
{
    boolean isBallComingUp = ball.getRotation() > 180;
    if (isBallComingUp)
    {
        int targetX = ball.getX();
        if (getX() < targetX - paddleSpeed) //slides 1
        {
            setLocation(getX() + paddleSpeed, getY());
        }
        else if (getX() > targetX + paddleSpeed)
        {
            setLocation(getX() - paddleSpeed, getY());
        }
    }
}
```

3.7. Sprint Review

1. We've made a game where you can control the bottom paddle using either 'a' & 'd' or left & right, and where the ball will bounce off of the walls and the paddle(s).
2. We created a paddle in the middle that will move from the left side of the screen and to the right side. Once it collides with the right wall, it will despawn and respawn at a random location on the left wall, at a certain point above the y-axis, so it doesn't immediately block the ball from bouncing from the bottom paddle.
3. We made it so that the ball can pass through the middle paddle, if it is travelling from the top of the screen and towards the bottom.
4. We used a SpongeBob theme for the images
5. For the sounds, we also used a SpongeBob theme, as we thought it would be more fun than just an assortment of random sounds and images.
6. At first, we implemented the speed increase for the ball to be whenever it had hit the paddles 10 times, but we then changed it to be 5 times instead, as we all agreed that 10 times made the game feel more sluggish and less fun than with the speed increase occurring more often.
7. We implemented a scoreboard which displays the current game level. Each time the ball reaches the required number of paddle hits, the ball speed increases and the game level increases as well. We later expanded the scoreboard, so it also displays the player's lives, score and number of bounces. This gives the player a better overview of their current progress during the game.
 - a) For the opening screen we thought about using a gif image but quickly learned that importing that class in Greenfoot meant we should probably understand a pre-written class containing close to 1000 lines of code of which we had zero ownership. Instead of spending valuable time from the project to read and interpret that premade class, we settled on using a picture and added a brief instruction.
 - b) For the game-over state, we thought it would be more fun to have two different states: One where you achieve a victory, which happens if you "score" 5 times, and one for defeat, which happens if you let the ball touch the floor 3 times. We eventually included a third game-over state, which will happen if the game reaches a speed of 12, which is controlled by the amount of times (5) the ball bounces/touches the bottom or the top paddle, but isn't affected by the middle paddle.
 - c) We made the middle paddle spawn in different sizes by using the `Greenfoot.getRandomNumber` for the width, but when we experimented with the height, we thought that because of the image we picked it didn't look right most of the time, so we kept the different size for the width and not the height.
 - d) For the opposing paddle, we mirrored the distance from the edge (our player-controlled paddle sits at $Y = 650$ out of 700 and the top paddle sits at 50 out of 700) as we thought it would eliminate any advantage/disadvantage for either paddle and the visuals are more pleasing that way.

- e) The actual optional assignment was that we should create and include two fields to keep track of the score of the game. Instead, we elected to expand on the scoreboard that already kept track of the current level to also contain the amount of lives the player had and the score, where the lives and the score are an inverted way of keeping track; The player wins if the score reaches 5 and the opponent wins if the lives reaches 0 and the game is a “draw” if the speed reaches level 12 where the screen explodes.
- g) In addition to the required and optional features, we added several features of our own to make the game more entertaining. The player has three lives, and losing a ball reduces the number of lives. The player can also score points by reaching the opposite side.

3.8. Sprint Retrospective

For the next project, we agreed that it would be an improvement if we had a schedule to adhere to, such as having assigned an estimated time to each part of the assignment.

It could be beneficial to us, if we kept track of when a part of the program/report was considered completed and how much time we expected to use compared to how much time was actually used. Certain points of the project took less time than we anticipated, and other parts were far more strenuous than we had thought it would be.

Our communication was solid, as we spent all our time either face to face or on a call online, where we shared our thoughts and asked each other when we had an idea, a problem or a suggestion.

7. Conclusion

The goal of this project was to develop the provided Greenfoot pong project into a functional and more entertaining game. We also added several optional and in game features to make the game more complete. We chose to give the game a SpongeBob theme, which helped make the project more personal and fun. During development, we encountered several problems, especially with collision detection, image hitboxes and file formats. Solving these problems helped us gain a better understanding of Greenfoot and debugging.

The project also taught us a lot about working together as a group. We worked both face to face and through discord, where we shared screens, discussed problems and helped each other understand the code. Overall, we are satisfied with the result because we managed to turn the original simple pong project into a playable game with our own theme, additional features and increasing difficulty. The project game gave us more experience with object-oriented programming, teamwork, testing and problem solving in Greenfoot.

References

- Google. (2024). *Gemini*. Retrieved September 14, 2026 from Gemini: gemini.google.com
- Greenfoot. (2024, October). *Greenfoot 3.9.0*. Retrieved September 14, 2026 from Greenfoot API: <https://www.greenfoot.org/files/javadoc/greenfoot/package-summary.html>
- Myinstants. (n.d.). *SpongeBob Sound Effects Collection (bo-womp, spongebob-fail, kaching, SadViolin, KrabsMoney)*. Retrieved September 14, 2026 from Myinstants: <https://www.myinstants.com/en/search/?name=spongebob>
- Nickelodeon. (1999). *SpongeBob SquarePants characters (SpongeBob, Plankton, Squidward, Mr. krabs)*. Retrieved September 14, 2026 from SpongeBob SquarePants Assets: <https://www.nick.com/>
- Nissen, S. (2026, September). *DragonsOrSnakes*. Retrieved September 14, 2026 from Github: <https://github.com/Muffen101/Greenfoot-project/tree/main/Greenfoot/DragonsOrSnakes>
- The Krusty Krab Theme (The Rake Hornpipe). (1999). KPM Music.